

FICHAS TECNICAS

ESTRUCTURAS ARBOREAS DE DATOS

BST Árbol Binario de Búsqueda

AVL Árbol Balanceado AVL

B/B+ Árbol B y B+

TRIE Árbol de Prefijos

HEAP Montículo Min / Max

DATOS DEL ESTUDIANTE

Karen Charmaine Rodas Equite

Carne: 202504327 | Curso: Estructuras de Datos | 2026

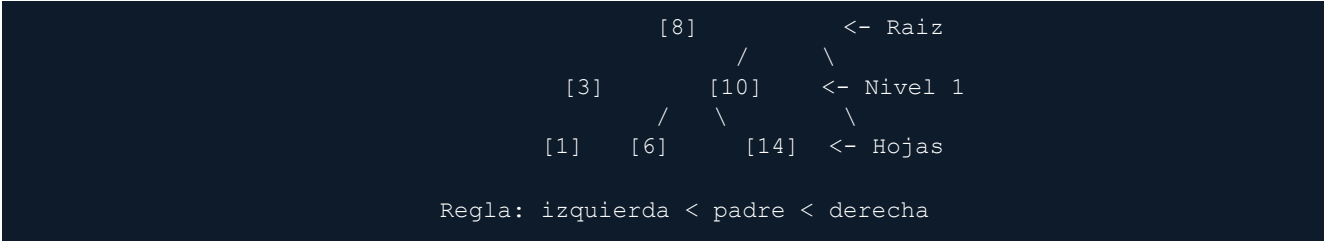
Árbol Binario de Búsqueda (BST)

Binary Search Tree | Estructura Fundamental de Datos

IDENTIFICACION

Árbol binario donde cada nodo cumple: todos los valores del subárbol IZQUIERDO son menores y todos los del subárbol DERECHO son mayores al nodo padre. No permite duplicados.

Visualizacion Conceptual (BST con raiz=8):



Análisis de Complejidad (Big O):

Operación	Peor Caso	Caso Promedio
Búsqueda	$O(n)$	$O(\log n)$
Inserción	$O(n)$	$O(\log n)$
Eliminación	$O(n)$	$O(\log n)$

Características Mecánicas:

Cada inserción compara el valor con el nodo actual: si es menor va a la izquierda, si es mayor a la derecha. Se repite hasta encontrar un espacio nulo. El orden NO está garantizado si el árbol no está balanceado.

VENTAJA PRINCIPAL

Búsqueda eficiente $O(\log n)$ en arboles balanceados. Simple de implementar y excelente para conjuntos de datos dinámicos con frecuentes inserciones y eliminaciones.

LIMITACION PRINCIPAL

Sin balanceo puede degenerar en $O(n)$ si los datos se insertan en orden ascendente o descendente. Se convierte en una lista enlazada.

CASO DE USO EN LA INDUSTRIA

Índices en bases de datos / Autocompletado

SQLite usa variantes de BST para índices. Los motores de búsqueda usan BST para indexar palabras clave en tiempo real. También presente en sistemas de autocompletado de texto.

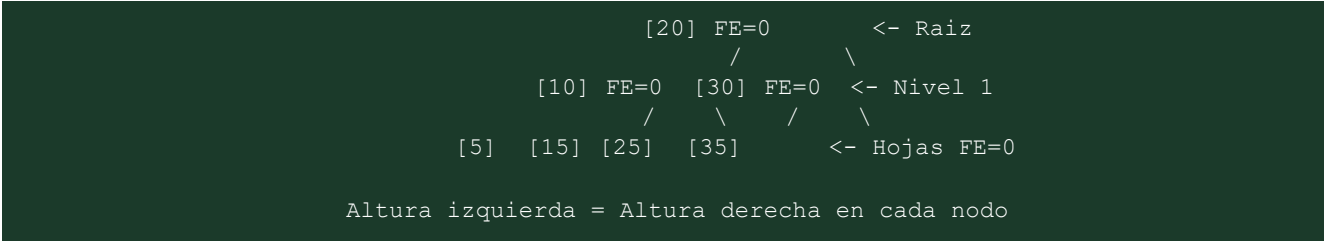
Árbol Balanceado AVL

Adelson-Velsky and Landis | Auto-balanceo garantizado

IDENTIFICACION

Árbol BST auto-balanceado. Después de cada inserción o eliminación calcula el Factor de Equilibrio ($FE = altura_izq - altura_der$). Si $|FE| > 1$, realiza ROTACIONES para restaurar el balance. Garantiza $O(\log n)$ en todas las operaciones.

Visualizacion Conceptual (todos los nodos con $FE=0$, arbol perfectamente balanceado):



Tipos de Rotaciones para restaurar balance:

LL - Rotación Simple Derecha

Cuando $FE = +2$ y FE del hijo izquierdo $= +1$

LR - Rotación Doble (Izq + Der)

Cuando $FE = +2$ y FE del hijo izquierdo $= -1$

RR - Rotación Simple Izquierda

Cuando $FE = -2$ y FE del hijo derecho $= -1$

RL - Rotación Doble (Der + Izq)

Cuando $FE = -2$ y FE del hijo derecho $= +1$

Análisis de Complejidad (Big O):

Operación	Peor Caso	Caso Promedio
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

VENTAJA PRINCIPAL

Garantiza $O(\log n)$ en TODAS las operaciones gracias al auto-balanceo. No degrada a $O(n)$ como el BST simple. Ideal para búsquedas frecuentes.

LIMITACION PRINCIPAL

Las rotaciones agregan overhead en escrituras. Mas complejo de implementar que un BST estándar. Cada nodo requiere almacenar su factor de equilibrio.

CASO DE USO EN LA INDUSTRIA

Java TreeMap / TreeSet / Sistemas de Trading de Alta Frecuencia

Java utiliza arboles Red-Black (pariente del AVL) en TreeMap y TreeSet internamente. Sistemas de trading de alta frecuencia usan AVL para ordenar ordenes de compra/venta en microsegundos garantizando siempre $O(\log n)$.

Árbol B / Árbol B+

B-Tree / B+ Tree | Optimizado para almacenamiento en disco

IDENTIFICACION

Árbol de búsqueda multinivel y auto balanceado donde cada nodo puede tener múltiples claves y múltiples hijos. El Árbol B+ almacena todos los datos en las hojas, que están enlazadas entre sí para recorridos secuenciales rápidos.

Visualización Conceptual (orden $t=2$, hasta 3 claves por nodo):

```

              [ 50 | 100 ]          <- Raiz
                /      |      \
          [10|20] [30|40] [60|70] [110|120] <- Internos
            |       |       |       |
        [datos] [datos] [datos] [datos] <- Hojas enlazadas

    B+: Hojas[1] <-> Hojas[2] <-> Hojas[3] (lista enlazada)
```

Arbol B vs Arbol B+ — Diferencias clave:

Árbol B	Árbol B+
Datos en nodos internos Y hojas	Datos SOLO en nodos hoja
Búsqueda puede terminar en nodo interno	Búsqueda SIEMPRE llega a las hojas
Hojas NO enlazadas entre si	Hojas enlazadas (lista doblemente enlazada)

Análisis de Complejidad (Big O):

Operación	Peor Caso	Caso Promedio
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

VENTAJA PRINCIPAL

Factor de ramificación alto reduce la altura del árbol. Ideal para sistemas de archivos y BD donde leer un bloque de disco trae muchas claves a la vez.

LIMITACION PRINCIPAL

Muy complejo de implementar. Las divisiones y fusiones de nodos en inserciones y eliminaciones generan overhead significativo.

CASO DE USO EN LA INDUSTRIA

PostgreSQL / MySQL InnoDB / NTFS (Windows) / ext4 (Linux)

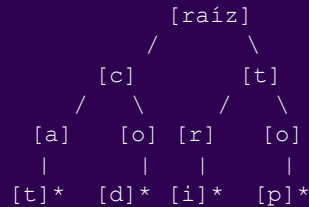
La mayoría de motores de bases de datos relacionales (PostgreSQL, MySQL InnoDB, Oracle) usan Arbol B+ para sus índices primarios. NTFS (Windows) y ext4 (Linux) los usan para indexar archivos en disco. MongoDB también los usa en sus índices.

Trie (Árbol de Prefijos)

Perfil Tree / Radix Tree | Especializado en cadenas de texto

IDENTIFICACION

Estructura de árbol donde cada nodo representa un CARACTER. Las palabras se forman recorriendo el camino desde la raíz hasta un nodo marcado como fin de palabra. Los prefijos compartidos ocupan los mismos nodos, ahorrando espacio.



* = marca fin de palabra
'cat' y 'cod' comparten el nodo [c] <- ahorro de espacio

Análisis de Complejidad (Big O) — m = longitud de la cadena de texto:

Operación	Peor Caso	Caso Promedio
Búsqueda	$O(m)$	$O(m)$
Inserción	$O(m)$	$O(m)$
Eliminación	$O(m)$	$O(m)$

Características Mecánicas:

Para insertar 'cat': desde la raíz se crea o sigue el nodo 'c', luego 'a', luego 't' y se marca 't' como fin de palabra. Para buscar 'cat' se sigue el mismo camino. Si en algún punto el carácter no existe, la búsqueda falla. La complejidad depende solo de la longitud m de la palabra, no del número total de palabras almacenadas.

VENTAJA PRINCIPAL

Búsqueda de palabras y prefijos en $O(m)$, independiente del número de palabras almacenadas. Ideal para autocompletado, correctores ortográficos y búsqueda por prefijo.

LIMITACION PRINCIPAL

Alto consumo de memoria si el alfabeto es grande. Cada nodo puede tener hasta 26+ hijos en inglés. Para Unicode puede ser inmanejable.

CASO DE USO EN LA INDUSTRIA

Google Search / Correctores Ortográficos / Routers de Red

El autocompletado de Google, el corrector de Word y los diccionarios de smartphones usan Tries para sugerir palabras en tiempo real. Los routers de red los usan para búsqueda de rutas IP (longest prefix matching) en tablas de enrutamiento.

Heap — Montículo Min / Max

Binary Heap | Estructura para prioridades y ordenamiento

IDENTIFICACION

Árbol binario COMPLETO que cumple la propiedad de montículo: en un MAX-HEAP cada padre es mayor o igual a sus hijos (la raíz es el máximo). En un MIN-HEAP cada padre es menor o igual (la raíz es el mínimo). Se implementa eficientemente como un arreglo.



Arreglo Min: [1, 3, 2, 7, 5, 6, 4]
Arreglo Max: [9, 7, 8, 4, 5, 6, 3]
 $hijo_izq = 2i+1$ | $hijo_der = 2i+2$ | $padre = (i-1)/2$

Análisis de Complejidad (Big O):

Operación	Peor Caso	Caso Promedio
Inserción	$O(\log n)$	$O(\log n)$
Eliminar raíz	$O(\log n)$	$O(\log n)$
Ver raíz (min/max)	$O(1)$	$O(1)$

Características Mecánicas:

INSERCIÓN (sift-up): Se agrega el nuevo valor al final del arreglo y se compara con su padre. Si viola la propiedad, se intercambian. Se repite hasta llegar a la raíz. ELIMINACIÓN (sift-down): Se reemplaza la raíz con el último elemento y se elimina el último. Luego se compara con los hijos y se intercambia con el menor/mayor. Se repite hasta que no haya violaciones.

VENTAJA PRINCIPAL

Acceso $O(1)$ al mínimo o máximo. Inserción y eliminación $O(\log n)$. Base del algoritmo HeapSort ($O(n \log n)$) y de las colas de prioridad.

LIMITACION PRINCIPAL

No permite búsqueda eficiente de elementos arbitrarios ($O(n)$). No es adecuado para búsquedas por rango ni para recorridos ordenados completos.

CASO DE USO EN LA INDUSTRIA

Algoritmo de Dijkstra / Sistemas Operativos / Java PriorityQueue

Dijkstra (GPS, redes) usa Min-Heap para extraer el nodo de menor distancia en cada paso. Los Sistemas Operativos usan heaps para el scheduler de procesos por prioridad. Java PriorityQueue y Python heapq son implementaciones directas de Min-Heap.